

Software Language Engineering (SLE): Labs

Erwan Bousse

Version 2026-2027, 2026-10-22

Table of Contents

1. 💻 Lab: first steps with langium . . . 1
   1.1. Prerequisites . . . 1
   1.2. Generating the example Persons DSL . . . 1
   1.3. Building the Persons DSL . . . 1
   1.4. Manual testing of the Persons DSL . . . 1
   1.5. Packaging the Persons DSL . . . 2
2. 💻 Lab: designing and implementing the Dialogue DSL abstract syntax . . . 5
   2.1. Abstract syntax for the Dialogue DSL V1 . . . 5
   2.2. Abstract syntax for the Dialogue DSL V2 . . . 6
3. 💻 Lab: designing and implementing the Dialogue DSL concrete syntax . . . 7
   3.1. Concrete syntax for the Dialogue DSL V1 . . . 7
   3.2. Concrete syntax for the Dialogue DSL V2 . . . 7
4. 💻 Lab: designing and implementing the Dialogue DSL semantics . . . 9
5. 🗒 Resource: Dialogue DSL specifications . . . 11
   5.1. Context . . . 11
   5.2. Objective: designing and implementing a *Dialogue DSL* . . . 12
   5.3. Milestones . . . 13

Chapter 1. 💻 Lab: first steps with langium

This first lab aims to play around with Langium to become a little more familiar with the Langium framework. The goal is not yet to implement your own DSL, but to make sure you have everything ready on your system, and to see how a Langium DSL can be compiled and packaged.

1.1. Prerequisites

Check you have the required prerequisites [https://gl.univ-nantes.io/sle/workbook/site/2026-2027/lectures/langium/#sec-prerequisites] on your system, and install them if needed.

1.2. Generating the example Persons DSL

To discover Langium, the Langium Yeoman generator is able to generate an example of very simple DSL by answering *Yes* to the question "*Generate example project*?". The example DSL allows to write program where you can simply declare *Persons* and then *Greetings* to these persons.

Using Yeoman, initialize a Langium project with the following answers:

- *Your extension name*: `persons-dsl`
- *Your language name*: `Persons DSL`
- *File extensions*: `.persons`
- *Your grammar entry rule name*: `Persons`
- *Include VSCode extension?*: Yes
- *Generate example project?*: **Yes**
- *Include CLI?*: Yes
- *Include language tests?*: No

Before going further, have a look at the files of the resulting project, especially:

- In the `packages` folder, observe that there are the three separate components of Langium DSL: `language` (with the DSL core components), `extension` (with the code to build a Visual Studio Code extension for the DSL), `cli` (with the code to build a CLI for the DSL).
- In the `packages/language/src` folder, have a quick look at the Langium grammar (`.langium`) of the language. This file implements the the syntax of the DSL using the Langium Grammar Language (which is a *metalanguage*).



Contrary to the method we will see in the course, this default Langium grammar does not use *declared types* to properly separate the abstract syntax from the concrete syntax. We will do things differently (and better)!

1.3. Building the Persons DSL

Open your new Langium project in Visual Studio Code, and **build the project**.

1.4. Manual testing of the Persons DSL

1.4.1. Manual testing of the Visual Studio Code extension

Run the Visual Studio Code extension of your DSL in development mode, then in the new Visual Studio Code window create a file name `alice_and_bob.persons`, and write the following content:

```
person Alice
person Bob

Hello Alice
```

Observe that:

- some words appear colored because they are recognized as *keywords* of the DSL;
- in the last line, you can CTRL+click on `Alice` and it brings you to the first line: this is a default navigation service allowing to browse a program using the *cross-references* it contains;
- if you type new content that does not follow the DSL grammar rules, the parser throws an error.



The editor you are testing was entirely generated by Langium based on the DSL grammar, without having to write a single extra line of code. This is a great benefit of language workbenches: using high-level abstraction from metalanguages (here the Langium Grammar Language), we are able to generate significant parts of a DSL implementation.

1.4.2. Manual testing of the CLI

- **Run the CLI of your DSL** to call the code generator of the DSL on the `alice_and_bob.persons` program. By default, the code generator of the Persons DSL is a *code generator* that compiles a Persons program into JavaScript. The resulting JavaScript file is simply a sequence of `console.log` statements that greet each person of the program.
- Run the produced JavaScript file using `node`: congratulation, you executed your first DSL program!

1.5. Packaging the Persons DSL



Close the second Visual Studio Code instance (where you tested the DSL) before continuing.

Let's first **package the Visual Studio Code extension:**

- Package the Visual Studio Code extension of your Langium project into a `.vsix` file.
- Install the `.vsix` file into your Visual Studio Code, then check that the extension is installed in your list of extensions.
- Try to open `alice_and_bob.persons`, and observe that you can the Persons DSL editor is successfully used to edit the file (color completion is visible and navigation services work). This time your DSL was *properly installed* in Visual Studio Code, instead of running in development mode.
- Uninstall the extension once you are done (to avoid cluttering your Visual Studio Code).

Then let's **package the CLI**:

- Package the CLI of your Langium project into `.tgz` npm packages.

- Install the CLI on your system, and try to use it using the newly installed command `persons-dsl-cli`.
- Uninstall the CLI once you are done:

```
npm uninstall persons-dsl-language persons-dsl-cli
```

Your are now able to distribute your DSL and install it in user enviroments!

Chapter 2. 💻 Lab: designing and implementing the Dialogue DSL abstract syntax

In all the following lab sessions we will design an implement a DSL named the **Dialogue DSL**. The proposed specifications of this DSL is available on a dedicated page: Chapter 5.



We will simulate an iterative process for the DSL design and implementation, hence the specifications describe different *versions* of the DSL. To make sure you can progress on different versions in parallel, you will have to create one separate Langium project *per version*.

2.1. Abstract syntax for the Dialogue DSL V1

In this first part we focus on the V1 of the Dialogue DSL.

2.1.1. Abstract syntax design

Propose an abstract syntax *design* for this DSL using a MOF metamodel.

2.1.2. Abstract syntax implementation

Generate the Dialogue DSL project

Using Yeoman, initialize a Langium project for your own *Dialogue DSL* with the following choices:

- *Your extension name*: `dialogue-dsl-v1`
- *Your language name*: `Dialogue DSL V1`
- *File extensions*: `.dialogv1`
- *Your grammar entry rule name*: here put the name of the *root concept* of your abstract syntax design
- *Include VSCode extension?*: Yes
- *Generate example project?*: **No**
- *Include CLI?*: Yes
- *Include language tests?*: No

Implement the abstract syntax

1. Implement your abstract syntax in a separate file `packages/language/src/dialogue-dsl-v1.abstractsyntax.langium`,
2. Import this new file at the beginning of the existing file `packages/language/src/dialogue-dsl-v1.langium`,
3. Call the Langium Generator and compile the project.



Do not touch the grammar yet, we will work on it in the next chapter!

Testing the abstract syntax by manually constructing ASTs

1. Create a file `packages/language/src/ast-test.ts`, and write a program that creates an AST for the

very first V1 dialogue tree shown in the specifications.

2. Open a *JavaScript Debug Terminal* and using a breakpoint and the debugger check that your constructer AST is correct.

2.2. Abstract syntax for the Dialogue DSL V2

In this first part we focus on the V2 of the Dialogue DSL.

2.2.1. Abstract syntax design

Using the metamodel prepared for the V1 of the Dialogue DSL as a starting point, propose an abstract syntax design for the V2.

2.2.2. Abstract syntax implementation

1. Generate a new separate Langium project for the V2 of the DSL named `dialogue-dsl-v2`, with `Dialogue DSL V2` as language name and `.dialogv2` as file extension.
2. Using the implementation of the abstract syntax of the V1 of the Dialogue DSL as a starting point, implement the abstract syntax of the V2.

Testing the abstract syntax by manually constructing ASTs

1. Create a file `packages/language/src/ast-test.ts`, and write a program that creates an AST for the very first V2 dialogue tree shown in the specifications.
2. Open a *JavaScript Debug Terminal* and using a breakpoint and the debugger check that your constructer AST is correct.

Chapter 3. 💻 Lab: designing and implementing the Dialogue DSL concrete syntax

In this lab we will design a concrete syntax for the **Dialogue DSL**. The proposed specifications of this DSL is available on a dedicated page: Chapter 5.



We will continue the simulated iterative process for the DSL design and implementation, hence the specifications describe different *versions* of the DSL. To make sure you can progress on different versions in parallel, you will have to continue working on separate Langium projects *for each version*.

3.1. Concrete syntax for the Dialogue DSL V1

In this first part we focus on the V1 of the Dialogue DSL.

3.1.1. Concrete syntax design

- Propose an concrete syntax *design* for this DSL using a mockup.
- Identify token types in your mockup.
- Create annotated mockups for each part of your design.

3.1.2. Concrete syntax implementation

In the `dialogue-dsl-v1` project:

1. Implement your concrete syntax in `packages/language/src/dialogue-dsl-v1.langium`,
2. Call the Langium Generator and compile the project.

Testing the concrete syntax

TODO

3.2. Concrete syntax for the Dialogue DSL V2

In this first part we focus on the V2 of the Dialogue DSL.

3.2.1. Concrete syntax design

Using the concrete syntax design prepared for the V1 of the Dialogue DSL as a starting point, propose an concrete syntax design for the V2.

3.2.2. Concrete syntax implementation

In the `dialogue-dsl-v2` project:

Using the implementation of the concrete syntax of the V1 of the Dialogue DSL as a starting point, implement the concrete syntax of the V2.

Testing the concrete syntax by manually constructing ASTs

TODO

Chapter 4. 💻 Lab: designing and implementing the Dialogue DSL semantics

TODO

Chapter 5. 🗒 Resource: Dialogue DSL specifications

5.1. Context

5.1.1. Dialogues in video games

In many sorts of video games, and in particular in adventure games [https://en.wikipedia.org/wiki/Adventure_game] or role-playing games [https://en.wikipedia.org/wiki/Role-playing_video_game], the player will be able to start a *dialogue* by speaking with one the characters of the game – also called the *non-player characters* of the game, or NPCs.

- A dialogue begins with the NPC telling something ("*Good morning, can I help you?*"), then a list of possible pre-written answers are presented to the player ("*I'm looking for help*" or "*No thanks, good bye*")
- The player can choose one of the possible answers, which will make the NPC tell something else. An important point is that what the NPC says *depends on the choice just made by the player*.



Figure 1. Example of ongoing dialogue in The Legend of Zelda – Breath of the Wild

In most video games, the possible dialogue choices offered to the player depend on the *context*, such as items possessed by the player, or the characteristics (*charisma*, *intelligence*) of the player character, or the past actions of the player, or the past choices made in the dialogue (for example one dialogue choice can make the NPC angry, and refuse to answer more questions).

5.1.2. Modeling dialogues as *dialogue trees*

In the video game industry, a typical way to encode a dialogue in a video game is in the form of a **dialogue tree** [https://en.wikipedia.org/wiki/Dialogue_tree]. The nodes of a dialogue tree represent all possible ways to unfold a dialogue, such as *lines* that a given NPC can tell the player in the dialogue, or the *choices* that the player can make.

A possible graphical syntax, is to represent each NPC line as a *node*, and each possible *player choice* by an arrow going to another node of the tree.

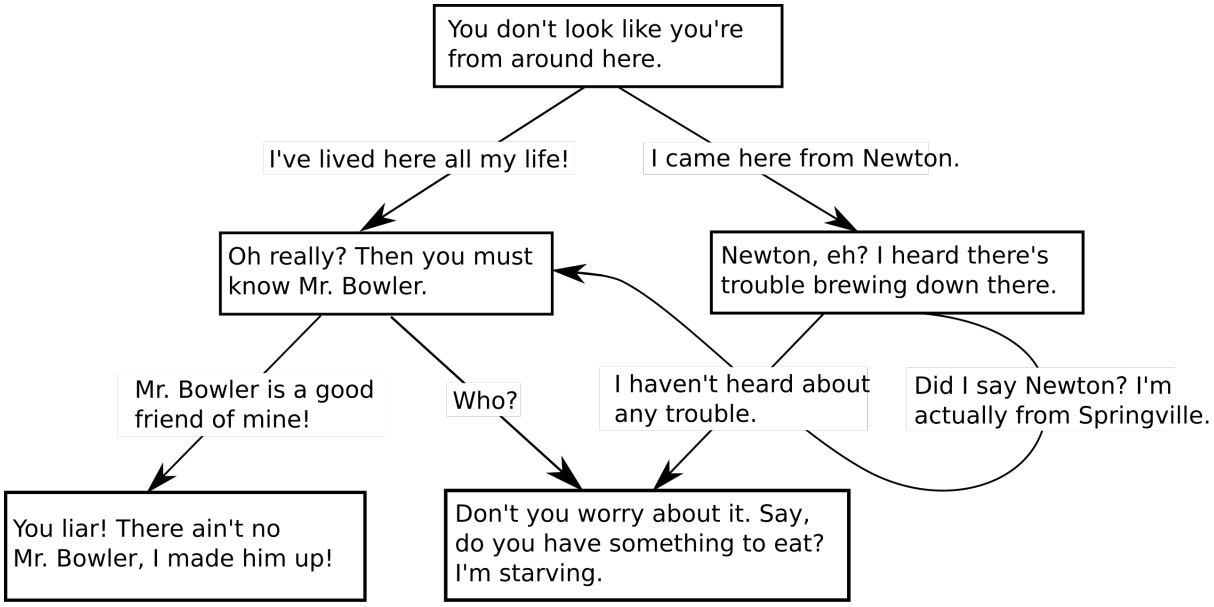


Figure 2. Example of simple dialogue tree. Each rectangle is an NPC line, and each arrow is a player choice. (source: Wikipedia)



This simplistic graphical syntax will not be sufficient to represent more complex dialogue trees in the later versions of the DSL you will create.



Despite the term, a "dialogue tree" is in fact *not* a tree data structure [https://en.wikipedia.org/wiki/Tree_(abstract_data_type)], because a player choice can point to any existing node of the tree. Therefore, technically a dialogue tree is more a kind of directed graph [https://en.wikipedia.org/wiki/Directed_graph]. This means it is OK for a dialogue tree to contain a *loop*, making the dialogue possibility infinite.

5.2. Objective: designing and implementing a *Dialogue DSL*

In the following series of practice works, you will have to design and implement a DSL called *Dialogue DSL* to represent dialogue trees. The goal is to allow this DSL is to have a ***textual* concrete syntax**, and to be usable by videogame experts that are *not* software engineers, despite having a basic understanding of what is coding.

We imagine this DSL to be part of a larger engineering context where the different core parts of a video game are made using DSLs that are directly put in the videogame experts hands:

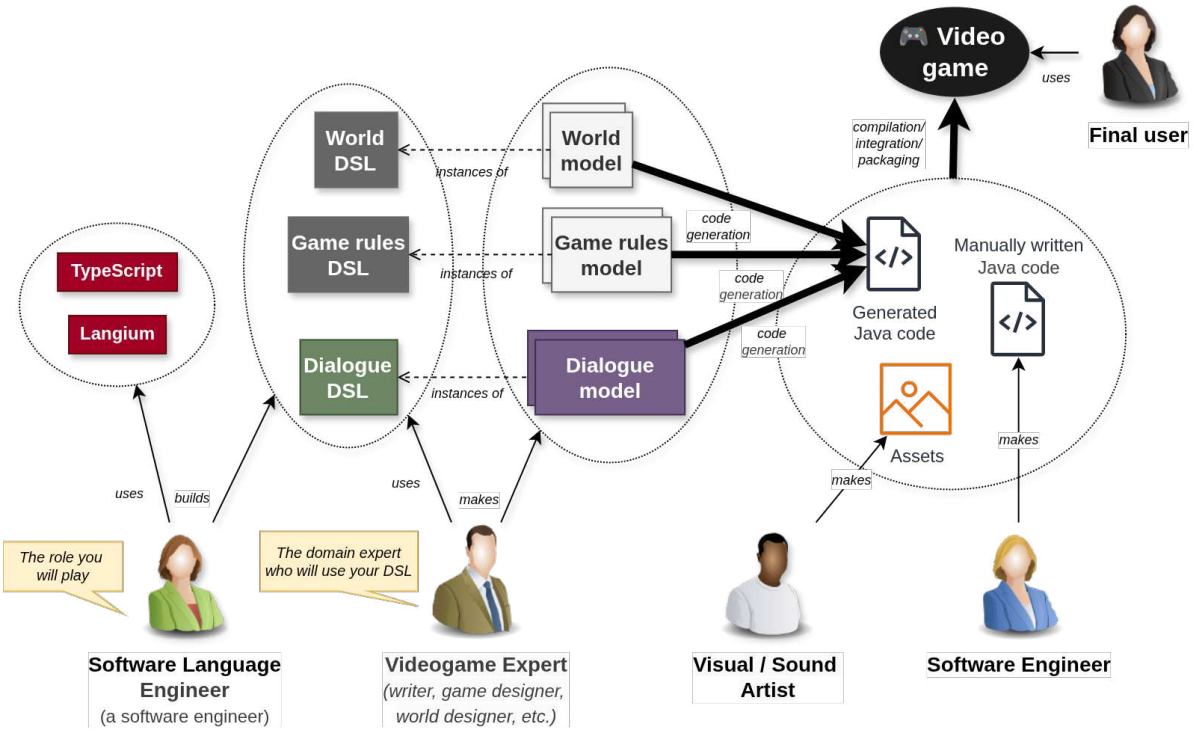


Figure 3. Context: engineering a video game through DSLs



We imagine that our Dialogue DSL will be mostly used by *writers*. Hence, in terms of **concrete syntax**, we would like the Dialogue DSL to *not* look like a programming language, and instead to look similar to natural language, or *movie scripts* [https://en.wikipedia.org/wiki/Screenwriting], or *dialogues in a novel* [https://en.wikipedia.org/wiki/Dialogue_in_writing].

5.3. Milestones

For this practice work, we will distinguish four different milestones you can reach with your DSL:

- **Version 1**: simple dialogue trees similar to the example shown previously.
- **Version 2**: we add to the DSL *flags* and *conditions*, allowing a dialogue to have a "memory" of what has been said, and to change the NPC lines and the available choices in consequence.
- **Version 3**: we add to the DSL *branch nodes*, allowing different directions to be taken in a dialogue based on the activated flags.
- **Version 4**: we allow the definition of *multiple dialogues* in the same model, and the flags are shared between dialogues (for example: if a first NPC gives an information, it can make visible a player choice when discussion with a second NPC).

Based on your motivation and your progress, you can iteratively first work on V1, then expand it into V2, then into V3, and so on.

5.3.1. Version 1: simple stateless dialogues

Syntax

The V1 of the Dialogue DSL should be able to represent dialogue trees similar to the Figure 2 shown above.

- A *dialogue tree* has a name, the name of the speaking NPC, and a starting line pronounced by the NPC. It is composed of a set of *nodes*, each node with one or zero NPC *line* , and many or zero *player choices*.
  - If a node has no player choices, then it directly has a pointer to the next NPC *line*.
  - If a node has no NPC line, then it directly allows the player to make a choice.
- An NPC *line* contains the text pronounced by the NPC.
- A *player choice* contains the text pronounced by the player, and a pointer to an existing node of the tree.

Example 1. V1 dialogue derived from The Secret of Monkey Island (1990)

Our running example will be the very first dialogue a player can encounter in the notorious video game *The Secret of Monkey Island* (1990). This dialog happens when discussing with an NPC named Mancomb in the *Scumm bar* near the start of the game.



Figure 4. Inside the Scumm bar, just before talking to Mancomb



Figure 5. Discussing with Mancomb

With the V1 version of the Dialogue DSL that we want, we would like to be able to represent a simplified version of this dialog which can be schematized like this:

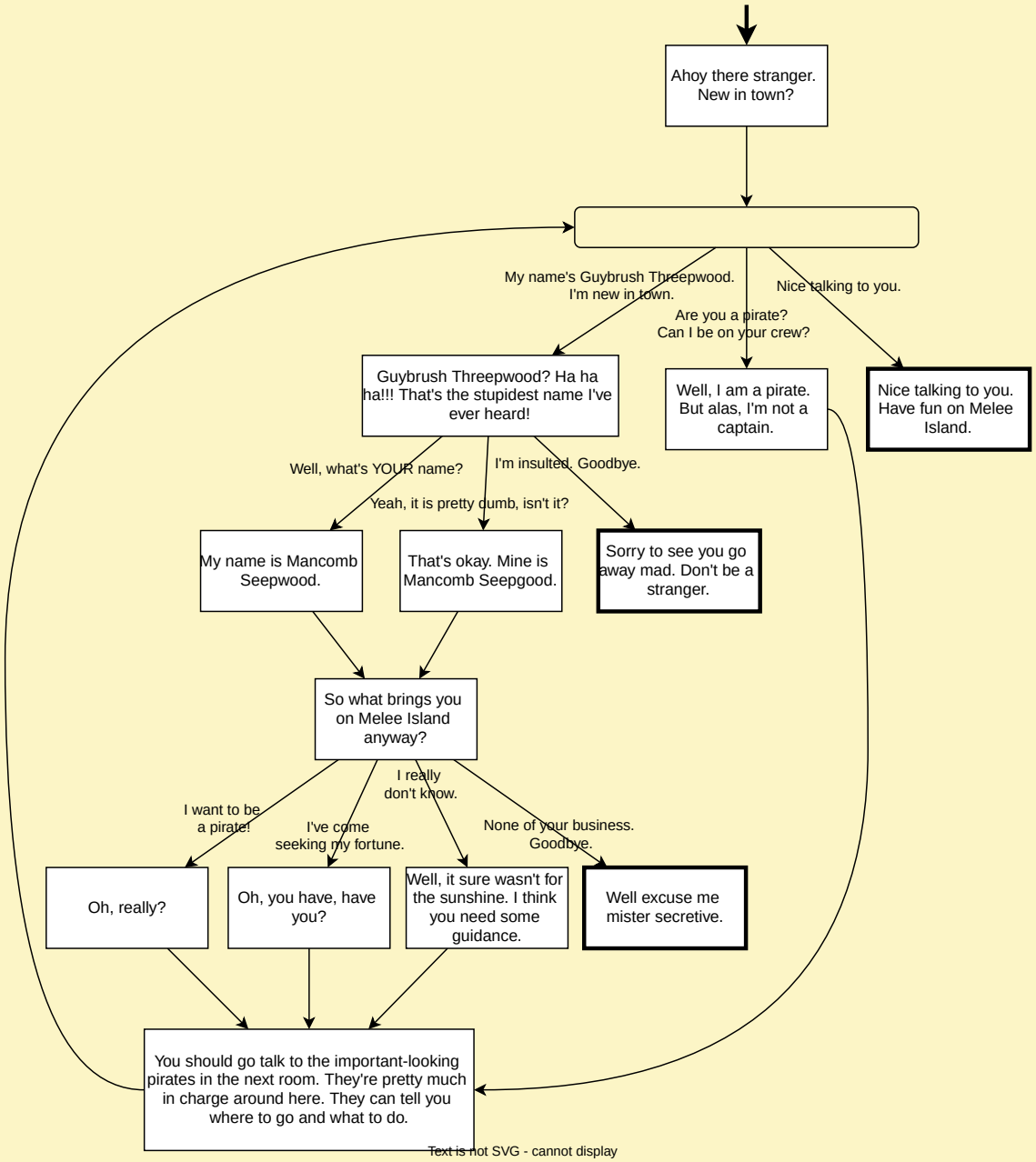


Figure 6. A simplified version of the dialogue with Mancomb



You can right-click the model and select "*Open image in new tab*" to display a version where the text can be selected and copy/paster.

We use here the following graphical concrete syntax:

- Regular *rectangles* are NPC lines.
  - Rectangles in *bold* show the NPC lines with no outgoing pointers (which end the dialogue).
  - Rectangles with a *hatch fill* are nodes without NPC lines
- *Arrows* depict either player choices (if there is a label), or the pointer after a NPC line without player choices.
- The *thick arrow* at the top shows the starting NPC line.



In this simplified version of the dialogue, it is possible to the same question *twice*, which was not possible in the original video game. With the V2 of the Dialogue DSL, we will design how to *remember* that a question was asked, to hide the question once it was asked.

Semantics

We would like to implement the semantics of the Dialogue DSL as a *code generator* that produces Java code. This Java code should contain a Java class with one Java method per NPC *line*, and a `start` method that calls the first NPC *line*. An NPC *line* method should print the NPC *line* and print all possible choices, then should expect a keyboard input to allow the player to choose.

For example, with the V1 of the DSL, the example dialogue tree showed in Figure 2 should lead to code similar to this:

```
import java.util.ArrayList;
import java.util.List;

public class MyDialogue {


    public void start() {
        line1();
    }

    public void line1() {
        // NPC line
        Util.display("You don't look like you're from around here");

        // Preparing list of all choices
        List<String> choices = new ArrayList<>();
        final String c1 = "I've lived here all my life!";
        choices.add(c1);
        final String c2 = "I came here from Newton.";
        choices.add(c2);

        // Print choices and ask answer
        Util.printChoices(choices);
        String choice = Util.askUserChoice(choices);

        // Move to next NPC line
        switch (choice) {
            case c1:
                line2();
                break;
            case c2:
                line3();
                break;
        }
    }

    public void line2() {

        // NPC line
```

```
        Util.display("Oh really? Then you must know Mr. Bowler.");

        // Preparing list of all choices
        List<String> choices = new ArrayList<>();
        final String c1 = "Mr. Bowler is a good friend of mine!";
        choices.add(c1);
        final String c2 = "Who?";
        choices.add(c2);

        // Print choices and ask answer
        Util.printChoices(choices);
        String choice = Util.askUserChoice(choices);

        // Move to next NPC line
        switch (choice) {
            case c1:
                line4();
                break;
            case c2:
                line5();
                break;
        }
    }

    public void line3() {

        // NPC line
        Util.display("Newton, eh? I heard there's trouble brewing down there.");

        // Preparing list of all choices
        List<String> choices = new ArrayList<>();
        final String c1 = "I haven't heard about any trouble.";
        choices.add(c1);
        final String c2 = "Did I say Newton? I'm actually from Springville.";
        choices.add(c2);

        // Print choices and ask answer
        Util.printChoices(choices);
        String choice = Util.askUserChoice(choices);

        // Move to next NPC line
        switch (choice) {
            case c1:
                line5();
                break;
            case c2:
                line2();
                break;
        }
    }

    public void line4() {

        // NPC line
        Util.display("You liar! There ain't no Mr. Bowler, I made him up!");
    }
```

```
    public void line5() {

        // NPC line
        Util.display("Don't you worry about it. Say, do you have something to eat? I'm
starving.");
    }
}
```

This assumes that an `Util` class is already available with the following methods:

```
import java.util.List;
import java.util.Scanner;

public class Util {

    static Scanner scanner = new Scanner(System.in);

    public static void printChoices(List<String> choices) {
        for (String choice : choices) {
            System.out.println("(" + (choices.indexOf(choice)+1) +") " + choice);
        }
    }

    public static String askUserChoice(List<String> choices) {
        int choice = -1;
        while (choice < 1 || choice > choices.size()) {
            System.out.println("\nEnter choice (1-" + choices.size() + "): ");
            String choiceString = scanner.nextLine();
            try {
                choice = Integer.parseInt(choiceString);
            } catch (NumberFormatException e) {
                System.out.println("Input must be a number.");
            }
        }
        return choices.get(choice-1);
    }

    public static void display(String s) {
        System.out.println(s);
    }

}
```

5.3.2. Version 2: state, flags and conditions

In version 2 (v2), we would like the Dialogue DSL to store a simple *internal state* allowing to remember lines heard by the player, and choices made by the player earlier in the dialogue. Using this state, the dialogue can be more dynamic, and "unlock" player options based on what was said earlier in the dialogue.

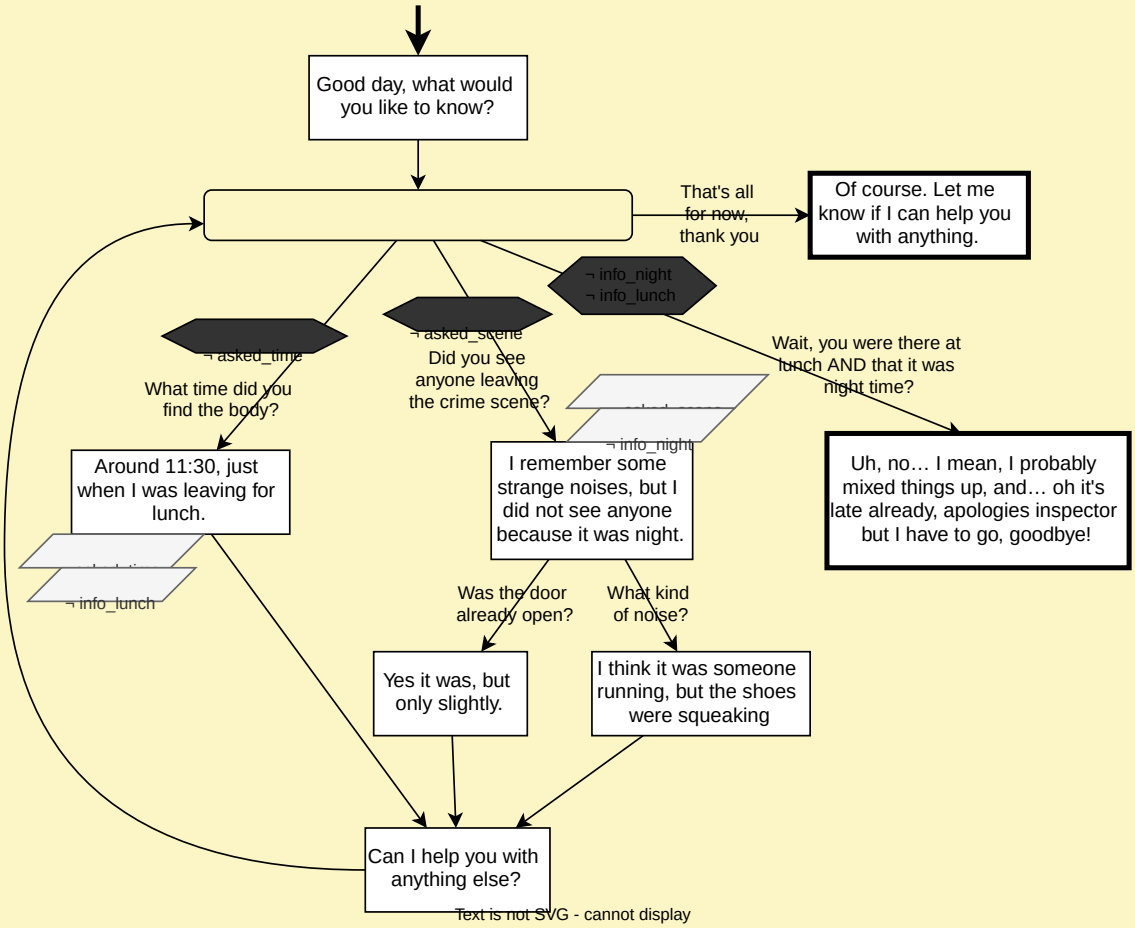
Syntax

We would like this state to be represented by **flags**, which are variables similar to booleans that can be either *set* or *unset*.

- We should be able to declare the list of all flags at the beginning of a program (by default all flags are *unset*).
- We should be able to *set* (or *unset*) one or multiple flags when an NPC line is reached.
- We should be able to *hide* (or *show*) certain choices based on whether one or multiple flags are set or unset.

Example 2. V2 simple dialogue

Let's imagine a very simple dialogue in a murder mystery video game, where we must ask questions to a witness:



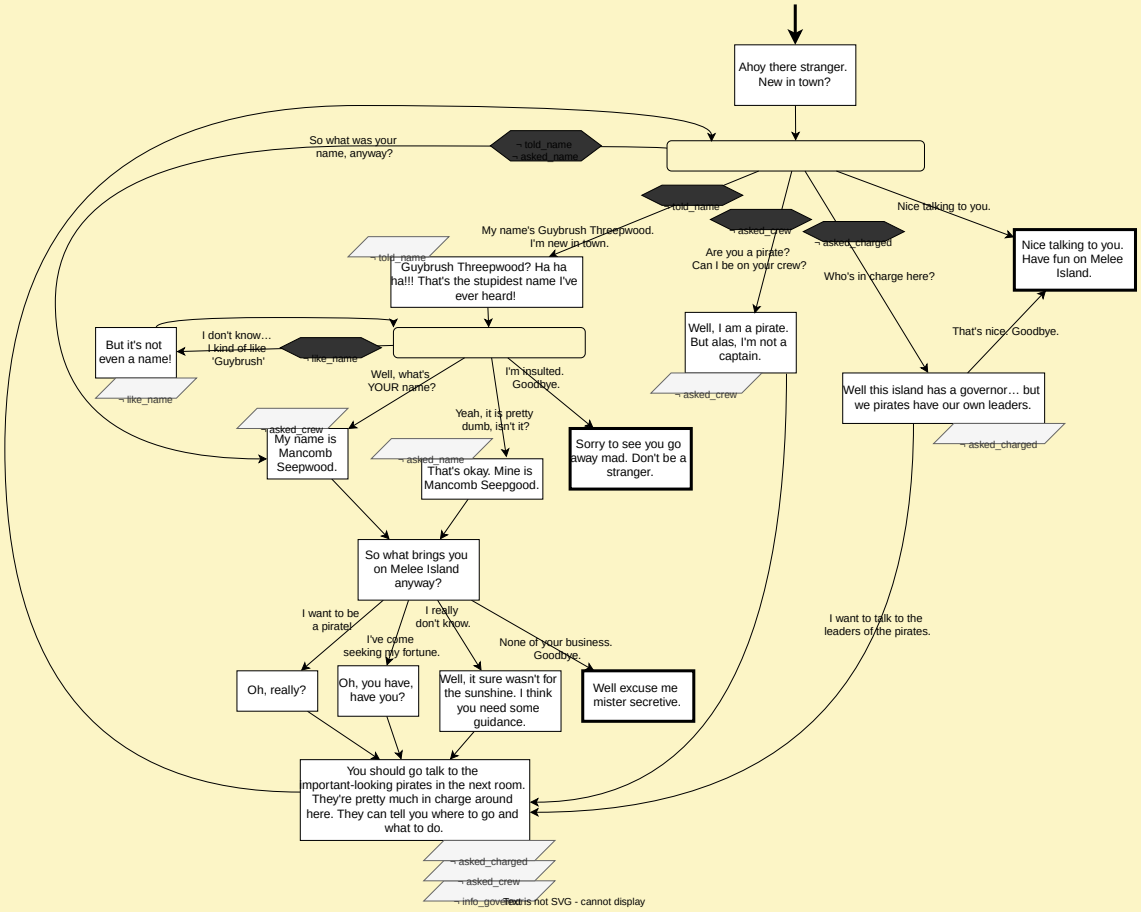
We introduce the following new notation:

- A *grey parallelogram* with the ✓ icon is placed where we want to *set a flag*, for example to remember that we asked a question.
- A *black long hexagon* represent a condition required to show a given player choice:
  - ✓ means the condition requires a flag to be *set*,
  - ✗ means the condition requires a flag to be *unset*.

Such conditions are used to only show a question if it was not asked already, or to make a new question available due to some acquired information earlier in the dialogue.

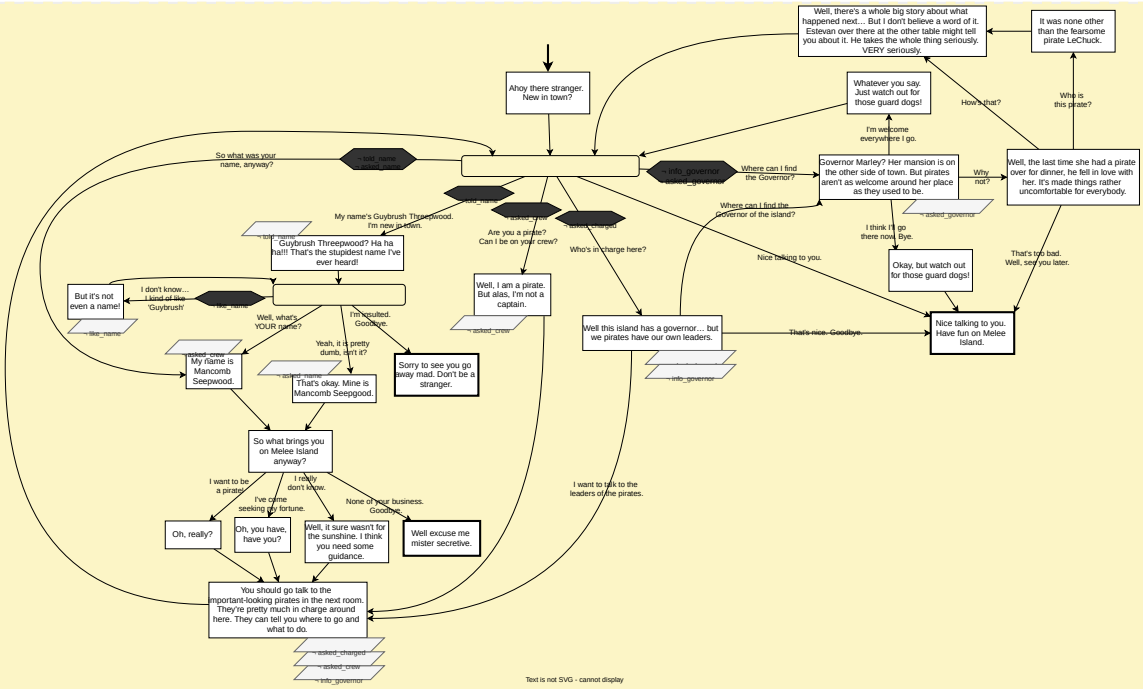
Example 3. V2 dialogue derived from The Secret of Monkey Island (1990)

We continue with the V1 dialogue taken from the video game *The Secret of Monkey Island* (1990). This time we would like to represent that we cannot ask the same question twice, and that some questions are only available under some conditions.



Example 4. More complete V2 dialogue derived from The Secret of Monkey Island (1990)

To be closer to the dialogue in the original video game, we can model the following more complete version (where we can ask about the Governor):





We are still not able to represent the full dialogue of the original game! We miss a concept allowing us to change the direction of the dialogue at any possible time, which will be added in the V3.

Semantics

You must improve the code generator of the V1 in order to generate appropriate code for the newly introduced concepts:

- For each possible *flag*, a corresponding *boolean variable* can be generated in the enclosing Java class:

```
boolean my_flag = false;
```

- When a flag must be *set*, the corresponding boolean variable can be assigned the value *true*:

```
my_flag = true;
```

- When a choice must be hidden because of flags, a Java conditional (`if`) can be generated to only consider the choice when the corresponding boolean variables have the right values:

```
if (my_flag) {
    choices.add(some_choice);
}
```